HermSec: A User-Friendly Repository Security Scanner with a Comparison of Static, Single-Agent, Multi-Agent, and Hybrid Review Modes

Seth Whenton
University of Passau
Passau, Germany

Ghazal Pouresfandiyar Borojeni
University of Passau
Passau, Germany

Abstract

HermSec is a desktop security scanner for local repositories. It is built to help a developer choose a project, inspect what kind of code it contains, prepare the right scanner tools, run the scan, and generate a readable report. The app includes a chat workflow, live progress cards, scanner settings, model provider settings, doctor checks, automations, and HTML/PDF/JSON reports.

This paper first presents HermSec as a product, then compares four review modes. Deep assisted scan runs static scanners and uses a model to explain scanner evidence. Single agent inspection lets one model inspect bounded code snippets without scanners. MoA (Mixture of Agents) inspection uses several specialist agents, a false-positive judge, and an aggregator without scanners. Scanner + MoA inspection runs scanners and MoA together, then uses a judge and final aggregator to validate both sets of findings.

This paper reports completed tests on our dataset of 20 projects and 184 labelled findings. The tested configurations were Deep assisted scan, Single Agent inspection, MoA Low, MoA High, Scanner + MoA Low, and Scanner + MoA High. The best F1 score was Scanner + MoA High with 0.7640. Single Agent reached 0.6462 and was faster. The result shows that the hybrid mode gave the strongest balance.

A second evaluation completed 24 mode-and-fixture runs using efficient non-US models through OpenCode Go. MoA Low had the best F1 score at 0.4762. Single Agent had the best precision at 0.6667 but low recall. Scanner + MoA High had the best recall at 0.5833 but also many false positives. The result shows that multi-agent review can reduce noise compared with scanner-heavy modes, but the hybrid mode still needs stronger candidate filtering.

Keywords

static analysis, software security, large language models, multi-agent systems, vulnerability detection, code review

1 Introduction

Developers need security feedback while they write code. Static application security testing tools can help because they scan source code without running the program [3, 7]. These tools are useful, but they also have limits. They need good rules, language support, and careful tuning. If the rules are weak, the tool can miss real bugs. If the rules are too broad, the tool can report too many false alarms [22].

HermSec is a desktop app that scans projects for security issues. It uses built-in rules and external tools such as Semgrep, Gitleaks, Trivy, Bandit, and other scanners [1, 15, 16, 18]. The app also supports model-assisted review. During testing, one clear problem appeared. Scanner-based reports found many issues, but they could become noisy. Model-only review was easier to read, but it missed more issues.

The main product idea is simple. A user should be able to pick a repository and ask HermSec to scan it. HermSec then inspects the project, checks the language and project files, chooses the scanner tools that fit that project, prepares or installs missing managed tools, runs the scan, and writes a report. This is important because a React project, Java project, Python project, and Rust project should not all need the same scanner setup.

This project studies a simple question: can agent review improve HermSec's scanner reports without losing too much coverage?

The first step was building a test dataset with 20 projects and 184 labelled findings across multiple languages. Each project had planted vulnerabilities with known CWE identifiers. This dataset was used to test how well scanners and MoA modes work together. The second step compared four review modes on a smaller controlled fixture set. Both evaluations are reported in this paper.

The paper compares four HermSec modes:

- **Deep assisted scan**: HermSec runs scanners first. The model then explains scanner evidence when possible.
- **Single agent inspection**: One model reads selected code evidence and writes findings. No scanners run in this mode.
- **MoA inspection**: Several specialist agents inspect the code. A judge checks possible false positives. An aggregator writes the final report.
- **Scanner + MoA inspection**: HermSec runs scanners and MoA review, then a judge and final aggregator validate both sets of findings.

This paper does not claim that agents are always better than scanners. The goal is smaller and clearer. It tests how the four modes behave on controlled examples and explains what should be improved next.

Figures 1 and 2 show parts of the HermSec app that were built during the project. The scan workflow asks the user to choose a mode, shows live progress, and then writes a short result summary. The settings screens let the user configure agent modes and model providers.

2 HermSec Product Overview

HermSec is not only a benchmark script. It is a desktop application for normal developers who want a simple way to check a local project. The main screen is a chat interface. The user selects a project folder, chooses a scan mode, and watches the scan progress in the chat. When the scan ends, HermSec writes report files and gives links to the report folder and PDF.

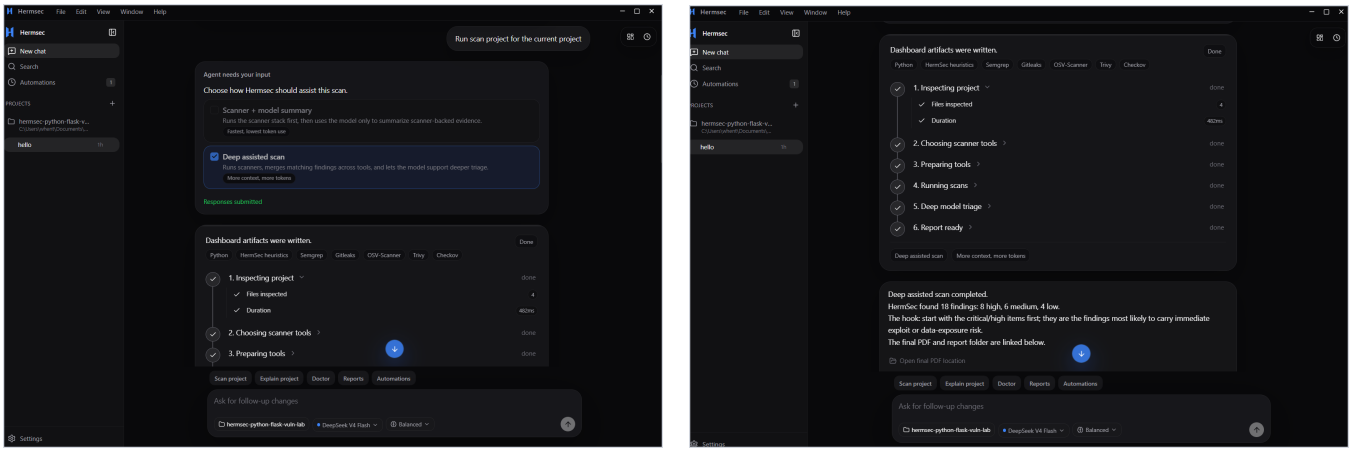

Figure 1: HermSec scan workflow. Left: the user chooses a scan mode and sees live progress. Right: the scan finishes and HermSec shows a short summary with report links.

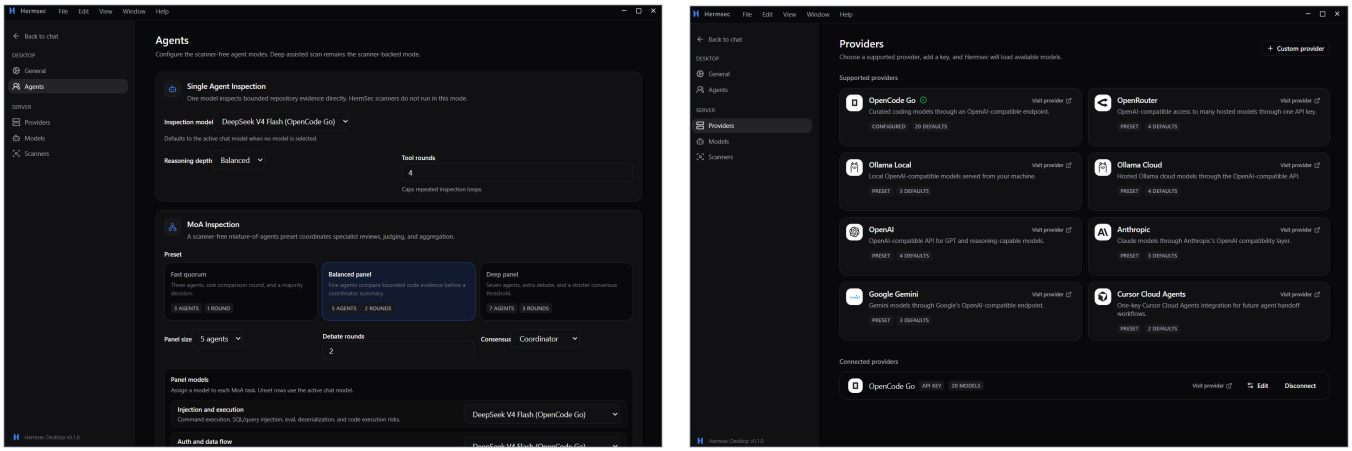

Figure 2: HermSec settings screens. Left: Agents settings for Single Agent and MoA inspection. Right: provider setup with OpenCode Go and other supported providers.

The product has four main surfaces. The home chat handles project selection, scan mode selection, progress, and final report links. Scanner settings show which tools are installed, enabled, missing, and relevant to the current project. Agent and provider settings let the user choose a provider, model, agent roles, judge behavior, and aggregation behavior. Doctor checks and automations help the user verify readiness before a scan or repeat a known workflow later.

HermSec also has a project inspection step. Before running a scan, it checks the repository files, languages, manifests, lockfiles, and configuration files. From this profile, HermSec chooses the scanners that make sense for the project. For example, a Node.js project can use JavaScript rules, npm audit style checks, secret scanning, OSV, and Trivy. A Python project can use Python rules, Bandit, pip-audit, OSV, and Trivy. This keeps the scan workflow smaller and avoids installing every possible tool for every project.

The scanner manager is part of the product. It shows which scanners are installed, missing, enabled, and used by the current project. Supported scanners can be installed into HermSec's managed tool directory instead of being forced into the user's global system path. Figure 3 shows this scanner settings screen.

Other product features include Doctor checks, model provider setup, model selection, agent settings, scan automations, live progress cards, and HTML/PDF/JSON reports. The Doctor checks confirm that scanner tools, internet connectivity, and provider setup are ready. The report system keeps the final result readable instead of only printing raw scanner output.

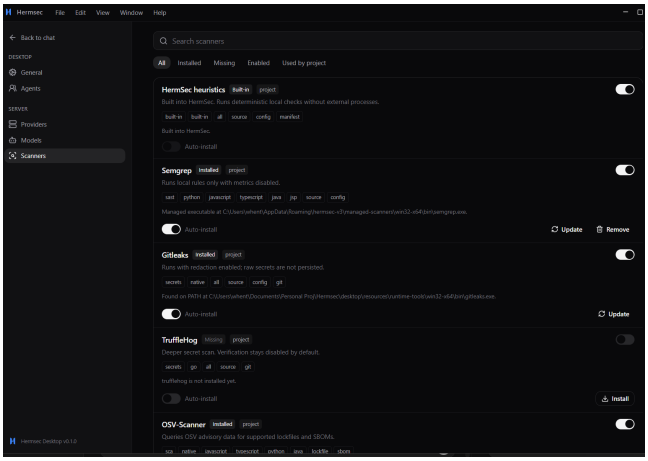

Figure 3: HermSec scanner settings. The app shows installed, missing, enabled, and project-relevant scanners.

3 Background and Related Work

3.1 Static Security Analysis

Static analysis checks code before the program runs. It is often used to find security problems such as injection, unsafe data flow, and dangerous API use [3, 7]. However, static tools must be tested with known examples. A tool may look strong in one project and weak in another project [22].

Security benchmarks help make this testing fair. OWASP Benchmark and NIST Juliet are examples of test suites with known bugs [2, 13]. This project uses the same idea, but with smaller local fixtures. The findings are labelled with CWE identifiers from MITRE CWE [9]. The tested vulnerability types also match common web security risks, such as injection, cross-site scripting, weak cryptography, hardcoded secrets, and unsafe configuration [12].

3.2 Scanner Harnesses

Many security tools combine several scanners. This is because one scanner usually does not cover every language or every type of vulnerability. HermSec follows this approach. Semgrep gives general static analysis coverage. Gitleaks finds secrets. Trivy finds dependency, secret, and configuration issues. Bandit improves Python coverage [1, 15, 16, 18].

This scanner approach is fast and useful. The main problem is that scanner outputs must be cleaned and merged. If HermSec only lists every scanner result, the user may see duplicates or low-value findings. For this reason, HermSec normalizes findings and stores evidence for each issue.

3.3 LLM Agents and MoA

Large language models can read code and explain risks, but they can also make mistakes [14]. Agent systems try to make model work more controlled by giving the model tools and steps. ReAct is one example of combining reasoning with actions such as searching or reading files [21].

Multi-agent work also shows that several model responses can sometimes improve the final answer [6, 17]. HermSec's MoA mode uses this idea for code security review. Different agents focus on different risk areas. Then a judge removes weak findings, and an aggregator writes the final report. This idea was also inspired by the configurable agent panels in Hermes Agent [10].

4 System Design

4.1 Deep Assisted Scan

Deep assisted scan is the scanner-based mode. HermSec first inspects the project. It chooses scanner tools, prepares them, runs them, and normalizes the findings. After that, the model can explain the scanner-backed evidence. If the model step fails, HermSec can still write a report from scanner findings. This is the closest mode to the original HermSec idea: a simple repository scanner that selects the right security tools for the project.

4.2 Single Agent Inspection

Single agent inspection does not run scanners. One model inspects the repository using safe read-only tools. It can list files, search code, and read snippets. It must report the file path, line or snippet, vulnerability category, evidence, confidence, and remediation. HermSec checks the evidence before adding the finding to the report.

4.3 MoA Inspection

MoA means mixture of agents. In this mode, HermSec runs a panel of specialist agents. The default roles include injection, secrets and supply chain, authentication, crypto and data protection, configuration and IaC, language and framework, API security, and database security. A false-positive judge checks the candidates. The aggregator then writes the final accepted findings.

HermSec supports two MoA configurations. MoA Low uses 3 specialist agents with a candidate limit of 60 per agent. MoA High uses 5 specialist agents with a candidate limit of 120. Higher configurations allow more agents to inspect more candidates, but they also cost more in model tokens and runtime.

4.4 Scanner + MoA Inspection

Scanner + MoA inspection is the hybrid mode added after the first tests. In this mode, scanners run and produce candidate findings. MoA agents also inspect the repository with bounded read-only code tools. Then a false-positive judge reviews both scanner findings and agent findings together. Finally, an aggregator writes the accepted and deduplicated findings into the normal HermSec report. The goal is to keep the broad coverage of scanners while using agents to reduce false positives and improve the final explanation. Figure 4 shows the execution flow for each mode.

4.5 Safety Limits

The agent modes are limited on purpose. They cannot run shell commands. They cannot install packages. They cannot execute project scripts. They cannot read outside the selected repository. HermSec also skips noisy folders such as `.git`, `node_modules`, build folders, and vendored code. These limits make the agent modes safer and easier to test. The complete harness flow, from

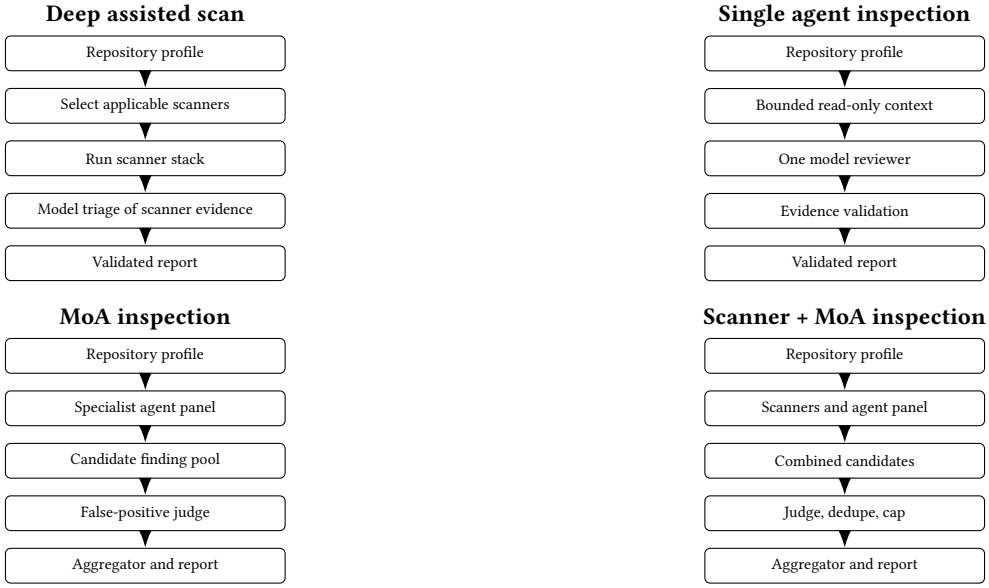

Figure 4: Execution diagrams for the four review modes. The diagrams separate scanner-backed review, one-agent direct review, multi-agent review, and the hybrid path that joins scanner and agent candidates before judging and aggregation.

repository inspection through mode selection to the final report, is shown in Figure 5.

5 Methodology

5.1 Research Questions

The study asks four questions:

RQ1 Which mode gives the best precision, recall, and F1 score?

RQ2 Does MoA reduce false positives compared with the scanner-based mode while finding more bugs than one agent?

RQ3 Does Scanner + MoA improve the balance by combining scanner coverage with agent filtering?

RQ4 What should be improved before testing on larger benchmarks?

5.2 Test Dataset

We tested HermSec on our dataset of 20 local projects. The dataset contains 184 labelled findings. It includes vulnerable and clean projects across JavaScript, TypeScript, Python, Java, PHP, Ruby, Go, Rust, C/C++, C#, Docker, and GitHub Actions style configuration. The findings cover injection, command execution, secrets, authentication, dependency risk, database risk, API risk, crypto issues, and configuration problems. CWE labels were used where possible [9]. The risk types also match common web security issues described by OWASP [12].

The second evaluation used four controlled fixtures with 12 expected findings total, following a methodology inspired by the OWASP Benchmark [13] for ground-truth-based evaluation. The fixture categories were:

- vulnerable Node.js/Express project,
- clean Node.js/Express project,
- vulnerable Python/Flask project,
- clean Python/Flask project.

The clean projects had no expected vulnerabilities. The vulnerable projects had six expected findings each. The answer keys and run records are stored in `results/ghazal-main-iteration6-final-score-summary`

5.3 Modes Tested

The final metric table uses the completed full-dataset runs. The tested configurations were:

- Deep assisted scan with `deepseek-v4-flash`.
- Single Agent with `deepseek-v4-flash`.
- MoA Low with three specialists, a judge, and an aggregator.
- MoA High with five specialists, a judge, and an aggregator.
- Scanner + MoA Low with scanners and the low MoA panel.
- Scanner + MoA High with scanners and the high MoA panel.

The MoA and Scanner + MoA model pool was `deepseek-v4-flash`, `mimo-v2.5`, `deepseek-v4-pro`, and `minimax-m3`. All model calls used OpenCode Go. Scanner + MoA Low completed 17 of 20 runs, so it is included as a secondary row. Scanner + MoA High completed all 20 runs and is the main hybrid result.

5.4 Harness Steps

The HermSec harness follows the same basic steps for every scan. First, it validates the local project path. Second, it walks the repository and records source files, languages, manifests, lockfiles, and ignored folders. Third, it chooses the scan plan. In scanner-based modes, it selects the scanners that match the project. In agent modes, it prepares bounded file, search, and snippet context. Fourth, it runs

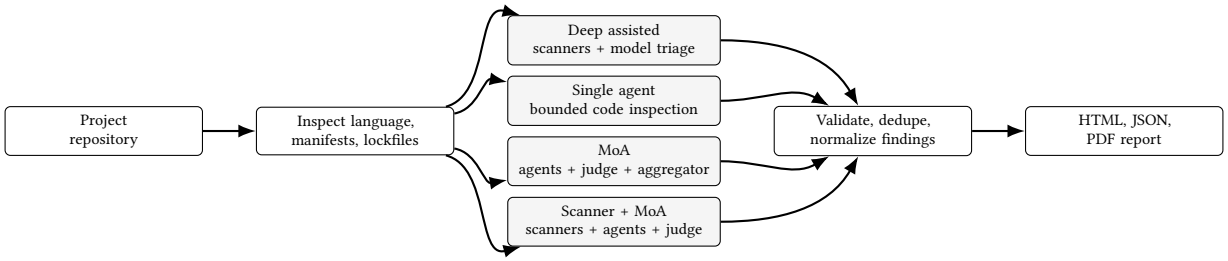

Figure 5: High-level HermSec harness flow. Every scan starts by inspecting the project. Then the selected mode runs and the results are validated, normalized, and written into the normal report template.

the selected mode with timeouts and progress events. Fifth, it normalizes findings into one schema. Finally, it writes the normal HermSec report.

The normal finding schema stores the title, category, severity, confidence, evidence, remediation, tool, rule ID, CWE, location, and fingerprint. This matters because scanner output and agent output must be scored and reported in the same format.

5.5 Model and Cost Choices

Another core idea in HermSec is cost control. Static scanners are the cheapest part because they run locally or through managed command-line tools. Model-assisted modes can become expensive because every agent reads project context and writes structured findings. For that reason, HermSec was designed to use efficient models first and reserve larger models for aggregation only when needed.

The model routing is intentionally open to efficient non-US providers and models exposed through OpenCode Go or similar provider adapters. `deepseek-v4-flash` is useful for this design because DeepSeek documents context caching and pricing behavior that can reduce repeated prompt-prefix cost when the same system prompts and report schemas are reused [4, 5]. Other efficient route models can be recorded in the result artifact's model policy. This matters for HermSec because Single Agent, MoA, and Scanner + MoA repeat similar instructions across fixtures.

MiMo V2.5 and MiniMax M3 are also relevant to the cost-control direction. MiMo V2.5 is described as an agent-capable long-context model with hybrid attention and KV-cache efficiency [19, 20]. MiniMax presents M3 as a coding and agentic model with long-context support [8].

The experiment used `deepseek-v4-flash` for Deep assisted and Single Agent. MoA and Scanner + MoA used `deepseek-v4-flash` and `mimo-v2.5` for specialist agents, `deepseek-v4-pro` for the judge, and `minimax-m3` for the aggregator.

5.6 Scoring

Each finding was compared with the ground truth by file path and CWE. A true positive means HermSec found a planted issue. A false positive means HermSec reported an issue that did not match the answer key. A false negative means HermSec missed a planted issue.

The metrics were calculated as follows:

$$Precision = \frac{TP}{TP + FP},$$
$$Recall = \frac{TP}{TP + FN},$$
$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

5.7 Problems Faced During Testing

There were a few relevant problems during testing. First, the Deep assisted explanation step hit provider timeout failures. This was improved by using smaller chunks and a longer watchdog. In the experiment run, two Deep assisted vulnerable rows still used fallback explanations because the model output did not pass evidence validation. Second, Scanner + MoA high failed when the false-positive judge received too many candidates in one request. This was fixed by batching the judge step and splitting failed batches. Third, scanner-heavy modes produced many findings outside the small answer key. Some of those findings may still be useful in a real project, but benchmark scoring counted only matches defined by the ground truth.

During app development, the main challenge was making the product workflow clear. HermSec needed to show live progress, manage scanners, keep model settings simple, and still write a normal report at the end. The solution was to keep one report format for all modes and add mode metadata such as agents used, judge counts, and source labels.

6 Results

Table 1 gives the main result. Figure 6 shows precision, recall, and F1. Figure 7 shows the true-positive and false-positive output count.

7 Discussion

The result shows that the working method improved the balance between coverage and false positives. Deep assisted found 85 true positives, but it still produced 119 false positives. This is expected because it depends strongly on scanner output.

Single Agent performed well for a cheaper mode. It found 126 true positives with 80 false positives and reached an F1 score of 0.6462. This is important because it shows that focused candidate tasks and evidence checks can make one model useful without running scanners.

Table 1: Main results from completed full runs on the dataset.

Mode	Runs	OK	TP	FP	FN	Precision	Recall	F1	Time (s)
Deep assisted	20	20	85	119	99	0.4167	0.4620	0.4381	715
Single Agent	20	20	126	80	58	0.6117	0.6848	0.6462	269
MoA Low	20	20	91	60	93	0.6026	0.4946	0.5433	1178
MoA High	20	20	84	40	100	0.6774	0.4565	0.5455	1560
Scanner + MoA Low	20	17	88	58	96	0.6027	0.4783	0.5333	4416
Scanner + MoA High	20	20	157	70	27	0.6916	0.8533	0.7640	2321

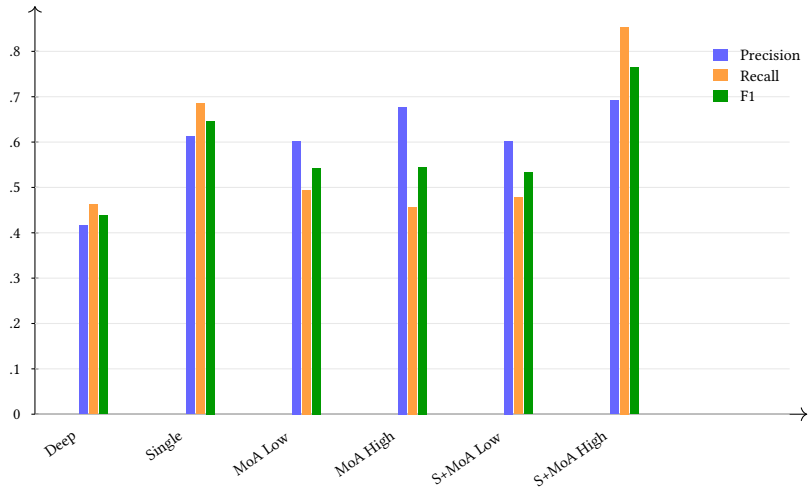

Figure 6: Precision, recall, and F1 for the completed full runs. Scanner + MoA High had the best F1.

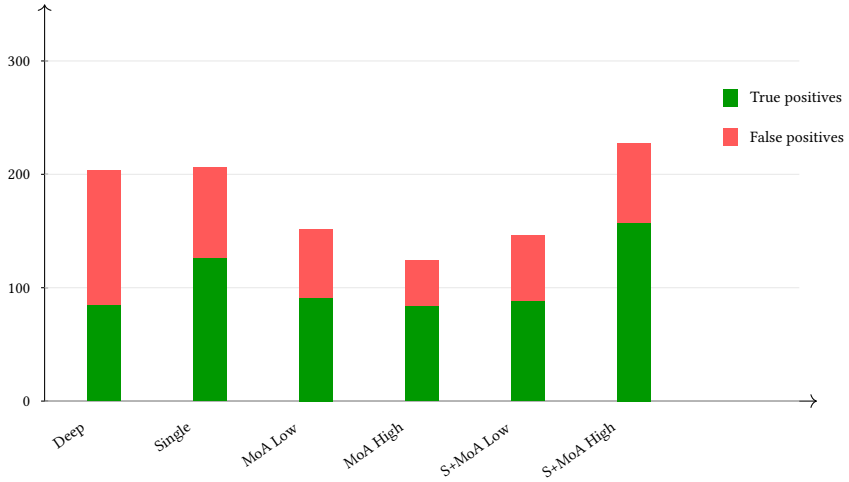

Figure 7: True-positive and false-positive counts. Scanner + MoA High found the strongest balance on this dataset.

MoA Low and MoA High were more balanced than Deep assisted, but they did not beat Single Agent. MoA Low reached an F1 score of 0.5433. MoA High reached 0.5455. This shows that adding more agents is not automatically better. The agents need good focused tasks and good rejection rules.

Scanner + MoA High gave the best result. It found 157 true positives, had 70 false positives, and reached an F1 score of 0.7640.

Scanner + MoA Low reached 0.5333, but only 17 of 20 runs completed successfully. This supports the main idea of HermSec. Scanners are useful for coverage, agents are useful for reasoning, and a judge/aggregator can merge the evidence into a stronger report.

7.1 Answer to RQ1

On the controlled fixture set, the answer depends on the goal. Single Agent had the best precision at 0.6667, but it only found 2 of 12 expected issues. Scanner + MoA High had the best recall at 0.5833, but it had many false positives. MoA Low had the best F1 score at 0.4762, so it was the best balanced mode in this run.

7.2 Answer to RQ2

For RQ2, MoA improved the balance compared with both Deep assisted and Single Agent. MoA Low found 5 true positives with 4 false positives. Deep assisted found 6 true positives, but it also reported 83 false positives. Single Agent had only 1 false positive, but it missed 10 expected issues. This means the MoA judge and aggregator helped reduce noise, but the system still lost some recall.

7.3 Answer to RQ3

For RQ3, Scanner + MoA did not beat MoA Low on F1 in this controlled run. Scanner + MoA High found the most true positives with 7 of 12, but it also produced 63 false positives. This means the hybrid preserved more scanner coverage, but the judge and aggregator did not reduce scanner noise enough yet.

7.4 Answer to RQ4

For RQ4, the next step is better coverage and more stable evaluation. HermSec still needs stronger language-specific rules and scanners for PHP, Go, Ruby, Rust, C#, C/C++, and configuration files. The agent modes should also be tested on larger public benchmarks. Deep assisted scan should continue reporting fallback reasons clearly. Scanner + MoA should use stricter scanner candidate filtering because too many scanner candidates made the final result noisy.

8 Threats to Validity

The dataset is useful, but it is still limited. It has 20 local projects and 184 labelled findings. The results may change on larger real repositories. The scoring is also based on matching findings to known labels, so matching rules can affect precision and recall.

The experiment used efficient models through OpenCode Go. Different model providers, different model versions, or different temperature settings may change the result. Exact token cost was not measured, so this paper only reports runtime and finding metrics.

This study is small. It used four controlled fixtures and 12 expected findings. The results show a useful direction rather than proving that the same behavior will happen on all projects.

The bugs are synthetic. Real projects can have more complex code and more complex data flow. The scoring method can also count a useful scanner finding as a false positive if it is not in the answer key. Finally, model behavior can change because of provider errors, prompt changes, timeout settings, and model updates.

9 Future Work

Future work should make HermSec more automatic and easier to use. The long-term product goal is a much smaller app with one main scan button. The app should inspect the project, choose tools, run the right mode, and write the report with less setup.

HermSec should also support email reporting. The agent could connect to a mailbox and send scan reports after new scans. This would help scheduled security checks.

Another useful future feature is an MCP server for HermSec. This would let agents inside editing tools call the HermSec CLI directly. For example, an editor agent could ask HermSec to scan the current project, wait for the result, and open the report without the user switching tools.

Another important feature is change detection. If a project has not changed since the last scan, HermSec should know that and avoid running the same full scan again. It could compare file hashes, git commits, package lockfiles, and stored report metadata. This would save time and reduce model tokens.

The hybrid mode also needs better runtime and cost control. Scanner + MoA High gave the best result, but it took a long time. Better scanner candidate limits, stronger judge prompts, better evidence checks, caching, and cost tracking should improve it.

10 Conclusion

HermSec is a desktop repository security scanner with scanner-backed and agent-backed review modes. The completed test used 20 projects and 184 labelled findings. Scanner + MoA High gave the best F1 score with 0.7640. Single Agent was the fastest strong mode with an F1 score of 0.6462.

The controlled run shows that no mode is perfect. Single Agent was the most precise, but it missed many issues. Scanner-heavy modes found more true positives, but they produced too much noise on the small answer key. MoA Low gave the best F1 score, so it was the best balanced mode in this experiment. Scanner + MoA High had the best recall, but it still needs better filtering before it can beat MoA on overall quality.

The main lesson is simple. Scanners are useful for coverage. Agents are useful for focused reasoning and explanation. The best direction for HermSec is not only scanners or only agents. The best direction is a careful hybrid design with focused tasks, strong evidence validation, and better false-positive control.

Acknowledgments

This work was developed for Security Insider Lab II.

AI Usage Disclosure

This project used AI assistance for dataset creation and evaluation. The test dataset of 13 vulnerable projects and 7 clean projects was created with help from Nemotron-3 Ultra [11]. The AI helped generate realistic vulnerable source code across 8 programming languages, create ground-truth files with CWE identifiers, and write the evaluation scripts.

The reason for using AI assistance was practical. Creating 20 complete test projects with realistic vulnerable code, proper project structure, and accurate ground-truth files across 8 languages would

take many weeks of manual work. The AI could generate consistent, well-structured code quickly, while the authors reviewed and validated the output.

References

[1] Aqua Security. 2026. Trivy: Scanner for Vulnerabilities, Misconfigurations, Secrets, and Licenses. https://github.com/aquasecurity/trivy. Accessed: 2026-06-27.

[2] Tim Boland and Paul E. Black. 2012. *Juliet 1.1 C/C++ and Java Test Suite.* Technical Report. National Institute of Standards and Technology.

[3] Brian Chess and Gary McGraw. 2004. Static Analysis for Security. *IEEE Security & Privacy* 2, 6 (2004), 76–79.

[4] DeepSeek. 2026. API Pricing. https://api-docs.deepseek.com/quick_start/pricing. Accessed: 2026-06-27.

[5] DeepSeek. 2026. Context Caching. https://api-docs.deepseek.com/guides/kv_cache. Accessed: 2026-06-27.

[6] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. https://arxiv.org/abs/2305.14325.

[7] V. Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium.*

[8] MiniMax. 2026. MiniMax-M3. https://www.minimax.io/models/text/m3. Accessed: 2026-06-27.

[9] MITRE. 2026. Common Weakness Enumeration. https://cwe.mitre.org/. Accessed: 2026-06-27.

[10] Nous Research. 2026. Hermes Agent. https://github.com/nousresearch/hermes-agent. Accessed: 2026-06-27.

[11] NVIDIA. 2026. Nemotron-3 Ultra: A Large Language Model for Multi-Domain Tasks. https://arxiv.org/abs/2606.15007. Accessed: 2026-06-27.

[12] OWASP Foundation. 2021. OWASP Top 10: The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/. Accessed: 2026-06-27.

[13] OWASP Foundation. 2026. OWASP Benchmark Project. https://owasp.org/www-project-benchmark/. Accessed: 2026-06-27.

[14] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy.*

[15] PyCQA. 2026. Bandit: Security Linter from PyCQA. https://bandit.readthedocs.io/. Accessed: 2026-06-27.

[16] Semgrep, Inc. 2026. Semgrep: Lightweight Static Analysis for Many Languages. https://semgrep.dev/. Accessed: 2026-06-27.

[17] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2024. Mixture-of-Agents Enhances Large Language Model Capabilities. https://arxiv.org/abs/2406.04692.

[18] Zachary Wilson. 2026. Gitleaks: Detect and Prevent Secrets in Git Repositories. https://github.com/gitleaks/gitleaks. Accessed: 2026-06-27.

[19] Xiaomi. 2026. Model Release: MiMo-V2 Series. https://mimo.mi.com/docs/en-US/updates/model. Accessed: 2026-06-27.

[20] XiaomiMiMo. 2026. Full-Pipeline Inference Optimization for MiMo-V2.5 Series. https://mimo.xiaomi.com/blog/mimo-v2-5-inference. Accessed: 2026-06-27.

[21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629.

[22] Misha Zitser, Richard Lippmann, and Tim Leek. 2004. Testing Static Analysis Tools Using Exploitable Buffer Overflows from Open Source Code. In *Proceedings of the 12th ACM SIGSOFT International Symposium on Foundations of Software Engineering.* 97–106.